DominaHub — Plano de Testes V0

Versão: 1.0 **Data:** Março 2026 **Produto:** DominaHub — Plataforma de Controle Financeiro Pessoal
Ambiente: Node.js 20 + PostgreSQL 15 + React 18

Sumário

1. Visão Geral da Plataforma
2. Escopo dos Testes
3. Arquitetura de Testes
4. Autenticação & Sessões
5. Transações
6. Contas Bancárias
7. Categorias
8. Orçamentos
9. Cartões de Crédito
10. Metas Financeiras
11. Micro Metas
12. Passivos (Dívidas)
13. Investimentos
14. Dashboard & Analytics
15. Forecast (Projeção Mensal)
16. Regras de Categorização Automática
17. Segurança de Dados
18. Ownership / IDOR Prevention
19. Faturamento & Planos
20. Matriz de Cobertura
21. Critérios de Aprovação V0

---

1. Visão Geral da Plataforma

DominaHub é um SaaS de controle financeiro pessoal com as seguintes capacidades:

- **Gestão de transações** — CRUD, recorrências, parcelamentos, rateios, anexos
- **Contas e cartões** — múltiplas contas, cartões de crédito com faturas
- **Orçamentos** — metas mensais por categoria com acompanhamento de gastos
- **Metas financeiras** — progresso com estimativa de conclusão
- **Investimentos** — posições e movimentos de ativos
- **Passivos** — dívidas com cronograma de pagamentos
- **Projeção mensal** — cenários otimista/base/conservador
- **Segurança** — MFA TOTP, sessões auditadas, criptografia em repouso

Stack técnica:

- Backend: Node.js 20, Express 4, Prisma 5, PostgreSQL 15, TypeScript
- Frontend: React 18, Vite, TailwindCSS
- Auth: JWT (access 15min) + refresh tokens httpOnly
- Pagamentos: Stripe

---

2. Escopo dos Testes

Incluído no V0

Tipo	Ferramenta	Escopo
Unitário	Jest + ts-jest	Services e helpers puros (sem banco)
Integração	Jest + Supertest	Endpoints HTTP com banco de testes
Manual	Checklist	Fluxos de UI críticos

Excluído do V0

- Testes de frontend automatizados (E2E com Playwright/Cypress)
- Testes de carga (k6 / Artillery)
- Testes de acessibilidade automatizados

---

3. Arquitetura de Testes

Estrutura de diretórios

```
backend/
  jest.unit.config.js           → config para testes unitários (sem banco)
  jest.integration.config.js    → config para integração (exige DATABASE_URL_TEST)
  src/
    __tests__/
      helpers/
        testDb.ts               → globalSetup: conecta ao banco de testes
        factories.ts            → funções utilitárias para criar dados de teste
      unit/
        encryption.test.ts
        goalsService.test.ts
        monthlyProjection.test.ts
        recurringService.test.ts
      integration/
        auth.test.ts
        transactions.test.ts
```

```
        ownership.test.ts
        budgets.test.ts
```

Scripts

```
# Testes unitários (rápidos, sem banco)
npm run test:unit

# Testes de integração (requer DATABASE_URL_TEST configurado)
npm run test:integration

# Todos os testes
npm run test:all

# Cobertura de código (apenas unitários)
npm run test:coverage
```

Variáveis de ambiente necessárias para integração

```
# .env.test
DATABASE_URL_TEST="postgresql://user:pass@localhost:5432/financial_control_test"
```

---

4. Autenticação & Sessões

Descrição

Sistema de autenticação stateful com JWT de curta duração (15min) + refresh tokens httpOnly (30 dias). Suporte a MFA TOTP opcional. Lockout automático após 5 falhas em 15 minutos.

Fluxo happy path

1. POST /auth/register → cria usuário + categorias padrão → retorna accessToken
2. POST /auth/login → valida credenciais → retorna accessToken + cookie refresh_token
3. POST /auth/refresh → renova accessToken usando cookie → token antigo invalidado
4. GET /auth/sessions → lista sessões ativas com isCurrent marcado
5. DELETE /auth/sessions/:id → revoga sessão → refresh com esse token falha

Casos de Teste — Registro

ID	Cenário	Entrada	Saída Esperada

AUTH-001	Cadastro válido	name, email único, senha forte	201 + accessToken
AUTH-002	Categorias padrão criadas	idem AUTH-001	categories.count > 0 para novo userId
AUTH-003	Email duplicado	email já cadastrado	409 Conflict
AUTH-004	Senha sem maiúscula	"lowercase1!"	400 + details com campo e mensagem
AUTH-005	Senha com menos de 8 chars	"Ab1!"	400 Validation error
AUTH-006	Email inválido	"not-an-email"	400 Validation error
AUTH-007	Nome muito curto (< 2 chars)	name: "A"	400 Validation error

Casos de Teste — Login

ID	Cenário	Entrada	Saída Esperada
AUTH-008	Credenciais válidas	email + senha corretos	200 + accessToken + cookie HttpOnly
AUTH-009	Cookie seguro	idem	Set-Cookie contém HttpOnly e SameSite=Strict
AUTH-010	Senha errada	senha incorreta	401 mensagem genérica (sem dica)
AUTH-011	Email inexistente	email não cadastrado	401 MESMA mensagem de AUTH-010 (anti-enumeração)
AUTH-012	Lockout após 5 falhas	5× senha errada → 6ª tentativa	429 na 6ª tentativa

Casos de Teste — MFA

ID	Cenário	Entrada	Saída Esperada
AUTH-013	Setup MFA	GET /auth/mfa/setup	URI otpauth:// válido
AUTH-014	Habilitar MFA	TOTP correto	200, mfaEnabled=true no banco
AUTH-015	Habilitar com código errado	TOTP inválido	400
AUTH-016	Replay attack	mesmo código usado 2×	400 TOTP_REPLAY_BLOCKED no audit
AUTH-017	Login com MFA ativo	credenciais corretas	200 com requiresMfa=true + mfaToken
AUTH-018	Verificar MFA	TOTP correto + mfaToken	accessToken válido
AUTH-019	mfaToken expirado (> 5min)	mfaToken antigo	401

Casos de Teste — Sessões & Tokens

ID	Cenário	Entrada	Saída Esperada
AUTH-020	Listar sessões	GET /auth/sessions	Array com isCurrent=true na sessão atual
AUTH-021	Revogar sessão	DELETE /auth/sessions/:id	204; refresh com esse token → 401
AUTH-022	Rotação de refresh token	POST /auth/refresh	Novo accessToken; token antigo inválido
AUTH-023	Troca de senha válida	currentPassword correto + newPassword forte	200
AUTH-024	Troca com senha atual errada	currentPassword incorreto	401
AUTH-025	Nova senha fraca na troca	newPassword sem símbolo	400

---

5. Transações

Descrição

Core do sistema financeiro. Transações podem ser simples, recorrentes (5 padrões), parceladas (crédito), rateadas (split) ou com anexos. Cada transação pertence a uma categoria e opcionalmente a uma conta.

Casos de Teste — CRUD Básico

ID	Cenário	Entrada	Saída Esperada
TXN-001	Criar INCOME	type=INCOME, amount, categoryId	201 + id
TXN-002	Criar EXPENSE	type=EXPENSE, amount, categoryId	201 + id
TXN-003	Listar por mês/ano	GET ?month=3&year=2026	Apenas transações do período
TXN-004	Filtrar por tipo	GET ?type=INCOME	Nenhum EXPENSE na resposta
TXN-005	Busca por descrição	GET ?search=Salário	Apenas transações com match
TXN-006	Atualizar descrição e amount	PATCH /:id	Valores refletidos na listagem
TXN-007	Deletar transação	DELETE /:id	204; não aparece mais na listagem

Casos de Teste — Paginação

ID	Cenário	Entrada	Saída Esperada
TXN-008	Paginar com limit=5	GET ?limit=5	data.length ≤ 5 + meta.total
TXN-009	hasMore correto	10 registros, limit=5, page=1	meta.hasMore=true
TXN-010	Limit acima do máximo	GET ?limit=999	Clampado a 100

Casos de Teste — Parcelamento (Cartão de Crédito)

ID	Cenário	Entrada	Saída Esperada
TXN-011	1 parcela	installments=1	1 transaction criada
TXN-012	3 parcelas	installments=3	3 transactions + 3 statements
TXN-013	Datas corretas	3 parcelas a partir de Jan	Jan, Fev, Mar
TXN-014	Distribuição de centavos	R$ 100/3	Parcela 1: R$ 33,34; demais: R$ 33,33
TXN-015	Formato da descrição	nome="Notebook", 3 parcelas	"Notebook (1/3)", "Notebook (2/3)", "Notebook (3/3)"

Casos de Teste — Split (Rateio)

ID	Cenário	Entrada	Saída Esperada
TXN-016	2 partes	splits=[{amount:40,cat1}, {amount:60,cat2}]	2 transactions com mesmo splitId
TXN-017	Categorias distintas por parte	split com 2 categoryIds	Cada transaction tem categoryId correto
TXN-018	Mínimo de 2 partes	splits=[{amount:100,cat}]	400 Validation error

Casos de Teste — Recorrência

ID	Cenário	Entrada	Saída Esperada
TXN-019	Criar template recorrente	isRecurring=true, MONTHLY	isRecurring=true no banco
TXN-020	Instância materializada	GET ?month=X&year=Y	Instância com parentId da template
TXN-021	Skip de instância	PATCH /:id/skip	isSkipped=true; não aparece no forecast
TXN-022	Pause de template	PATCH /:id/pause	isPaused=true; sem instâncias futuras

Casos de Teste — Anexos

ID	Cenário	Entrada	Saída Esperada
TXN-023	Adicionar anexo	POST /attachments com base64	201 + id
TXN-024	Criptografia em repouso	criar anexo	dataUrl no banco começa com "enc:"
TXN-025	Leitura descriptografada	GET /attachments/:id	dataUrl retornado é texto legível (base64 original)
TXN-026	Deletar anexo	DELETE /attachments/:id	204 + entrada em auth_events

---

6. Contas Bancárias

Descrição

Contas representam containers de dinheiro (corrente, poupança, carteira, etc.). O saldo é calculado dinamicamente a partir do saldo inicial + transações.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
ACC-001	Criar conta com saldo inicial	name, type=CHECKING, initialBalance=1000	201 + id
ACC-002	Saldo calculado corretamente	initialBalance=1000, EXPENSE=200, INCOME=500	balance=1300
ACC-003	Listar contas do usuário	GET /accounts	Apenas contas do userId
ACC-004	Atualizar nome e cor	PATCH /:id	Valores refletidos
ACC-005	Deletar conta sem transações	DELETE /:id	204
ACC-006	Deletar conta com transações	DELETE /:id	Transações também removidas (cascade)

---

7. Categorias

Descrição

Categorias organizam transações e orçamentos. Podem ser INCOME ou EXPENSE. Cada usuário tem categorias padrão criadas automaticamente no registro.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
CAT-001	Categorias padrão no registro	POST /auth/register	count > 0
CAT-002	Criar categoria EXPENSE	name, type=EXPENSE, color	201 + id
CAT-003	Criar categoria INCOME	name, type=INCOME	201 + id
CAT-004	Listar categorias	GET /categories	Apenas do userId
CAT-005	Atualizar nome/cor/ícone	PATCH /:id	Valores atualizados
CAT-006	Deletar categoria sem budget	DELETE /:id	204
CAT-007	Deletar categoria com transações	DELETE /:id	Depende da regra de negócio (400 ou cascade)

---

8. Orçamentos

Descrição

Orçamentos definem limites mensais de gasto por categoria EXPENSE. O campo `spent` é calculado dinamicamente. A criação é idempotente (upsert por userId+categoryId+month+year).

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
BDG-001	Criar orçamento para EXPENSE	categoryId (EXPENSE), amount, month, year	200 ou 201
BDG-002	Rejeitar orçamento para INCOME	categoryId (INCOME)	400
BDG-003	Upsert: segunda criação atualiza	mesmo categoryId+month+year, amount diferente	amount atualizado, count=1

BDG-004	Spent soma EXPENSEs do período	2 transações no mês: R$100 + R$200	spent ≥ 300
BDG-005	Percentual calculado	amount=500, spent=250	percentage=50
BDG-006	isCardPayment excluído do spent	transação isCardPayment=true	NÃO soma no spent
BDG-007	Deletar orçamento	DELETE /:id	204
BDG-008	Orçamento de outro usuário inacessível	DELETE /:id de userB usando tokenA	404

---

9. Cartões de Crédito

Descrição

Cartões de crédito geram faturas mensais (CardStatement). Transações parceladas criam entries na fatura de cada mês. O pagamento da fatura gera uma transação `isCardPayment=true` para não distorcer orçamentos.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
CC-001	Criar cartão	name, limit, closingDay, dueDay	201 + id
CC-002	Listar cartões do usuário	GET /credit-cards	Apenas do userId
CC-003	Fatura gerada para o mês	transação no cartão no mês X	statement para mês X criado
CC-004	Transação parcelada gera N faturas	3 parcelas	3 statements, uma por mês
CC-005	Total da fatura correto	2 transações no mês: R$200 + R$300	statement.totalAmount=500
CC-006	Pagar fatura	POST /statements/:id/pay	isCardPayment=true; fatura marcada como paga
CC-007	Acesso a fatura de outro usuário	GET /credit-cards/:otherId/statements	404

---

10. Metas Financeiras

Descrição

Metas vinculam uma conta bancária e calculam progresso dinamicamente. O status (ON_TRACK, BEHIND, VERY_BEHIND, COMPLETED) é determinado comparando estimatedCompletion com targetDate.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
GOAL-001	Criar meta com conta vinculada	name, targetAmount, linkedAccountId	201 + id
GOAL-002	Progress capped at 100%	currentAmount > targetAmount	progress=1.0
GOAL-003	Status COMPLETED	progress ≥ 1.0	status="COMPLETED"
GOAL-004	Status ON_TRACK	estimatedCompletion ≤ targetDate	status="ON_TRACK"
GOAL-005	Status BEHIND	estimatedCompletion > targetDate	status="BEHIND"
GOAL-006	Status VERY_BEHIND	monthlyContribution ≤ 0	status="VERY_BEHIND"
GOAL-007	Sem conta vinculada	linkedAccountId=null	currentAmount=0, status=VERY_BEHIND
GOAL-008	estimatedCompletion=null quando completo	progress=1.0	estimatedCompletion=null
GOAL-009	monthlyContribution=0 com < 2 meses	histórico < 2 meses completos	monthlyContribution=0
GOAL-010	Deletar meta	DELETE /:id	204

---

11. Micro Metas

Descrição

Checkpoints dentro de uma meta maior. Têm valores alvo menores e contribuem para o progresso da meta pai.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
MG-001	Criar micro meta	name, targetAmount, goalId	201 + id

MG-002	Listar micro metas de uma meta	GET /goals/:goalId/micro-goals	Apenas as do goalId
MG-003	Marcar como concluída	PATCH /:id com completedAt	completedAt definido
MG-004	Micro meta de outro usuário	GET /:id de userB usando tokenA	404

---

12. Passivos (Dívidas)

Descrição

Registra dívidas com saldo devedor e parcelas. O sistema calcula o progresso de pagamento.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
LIA-001	Criar passivo	name, totalAmount, monthlyPayment	201 + id
LIA-002	Registrar pagamento	POST /liabilities/:id/payments com amount	Saldo devedor reduzido
LIA-003	Listar passivos do usuário	GET /liabilities	Apenas do userId
LIA-004	Deletar passivo	DELETE /:id	204
LIA-005	Passivo de outro usuário inacessível	PATCH /:idDeB usando tokenA	404

---

13. Investimentos

Descrição

Posições de investimento agrupam ativos (ações, FIIs, renda fixa). Movimentos registram compras/vendas/dividendos.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
INV-001	Criar posição	name, ticker, type, currentValue	201 + id
INV-002	Registrar movimento de compra	type=BUY, quantity, price	saldo atualizado
INV-003	Listar posições	GET /investment-positions	Apenas do userId

INV-004	Atualizar valor atual	PATCH /:id com currentValue	Valor atualizado
INV-005	Posição de outro usuário inacessível	DELETE /:idDeB usando tokenA	404

---

14. Dashboard & Analytics

Descrição

Endpoint que agrega dados para a tela principal: net worth, resumo mensal, gastos por categoria.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
DASH-001	Net worth com contas	GET /dashboard	netWorth = soma de saldos das contas
DASH-002	Resumo mensal correto	transações do mês	totalIncome e totalExpense corretos
DASH-003	Gastos por categoria	transações em 3 cats diferentes	3 entradas no breakdown
DASH-004	Dashboard isolado por usuário	userA e userB com dados distintos	Cada um vê apenas seus dados

---

15. Forecast (Projeção Mensal)

Descrição

Calcula projeção de receitas e despesas para o mês atual. Usa média diária dos últimos 30 dias para estimar despesas variáveis. Gera 3 cenários (otimista, base, conservador).

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
FORE-001	daysInMonth correto	ano=2025, mês=2	28
FORE-002	daysInMonth ano bissexto	ano=2024, mês=2	29
FORE-003	Despesas fixas isoladas	transação isRecurring=true	incluída em fixedExpenses

FORE-004	Despesas variáveis isoladas	transação sem parentId	incluída em variableExpenses
FORE-005	Rolling 30-day avg	R$3.000 em 30 dias	dailyVariableAvg=100
FORE-006	estimatedVariableExpense	avg=100, daysRemaining=10	estimatedVariableExpense=1000
FORE-007	Cenário pessimista	variáveis estimadas	1.3× estimatedVariableExpense
FORE-008	Cenário otimista		apenas despesas fixas (sem variáveis estimadas)
FORE-009	Forecast isolado por usuário	2 usuários com dados distintos	Cada um vê apenas seus dados

---

16. Regras de Categorização Automática

Descrição

Regras baseadas em padrões de texto que categorizam transações automaticamente na importação.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
RULE-001	Criar regra	pattern, categoryId, priority	201 + id
RULE-002	Regra aplica ao importar	transação com texto que dá match	categoryId preenchido automaticamente
RULE-003	Prioridade respeitada	2 regras dão match	Regra de maior prioridade vence
RULE-004	Deletar regra	DELETE /:id	204
RULE-005	Regra de outro usuário inacessível	PATCH /:idDeB usando tokenA	404

---

17. Segurança de Dados

Descrição

Testa a stack de segurança: criptografia em repouso, redação de PII nos logs, auditoria.

Casos de Teste — Criptografia

ID	Cenário	Entrada	Saída Esperada
SEC-001	encrypt() retorna prefixo enc:	encrypt("hello")	String começa com "enc:"
SEC-002	Round-trip encrypt/decrypt	decrypt(encrypt(x))	=== x
SEC-003	IV aleatório por chamada	encrypt(x) chamado 2×	Ciphertexts diferentes
SEC-004	Backward compat: plain text	decrypt("texto-sem-prefixo")	Retorna "texto-sem-prefixo"
SEC-005	Adulteração detectada	ciphertext modificado	Exception lançada
SEC-006	Anexo cifrado no banco	POST /attachments	dataUrl no banco ≠ dataUrl original
SEC-007	Anexo descriptografado na leitura	GET /attachments/:id	dataUrl retornado = dataUrl original

Casos de Teste — Logger & Auditoria

ID	Cenário	Entrada	Saída Esperada
SEC-008	Senha não aparece nos logs	logger.error("err", {password: "123"})	Log mostra [REDACTED]
SEC-009	Token não aparece nos logs	{token: "abc"}	[REDACTED] no log
SEC-010	Audit de deleção registrado	DELETE /transactions/:id	Entrada em auth_events
SEC-011	Audit de login registrado	POST /auth/login	Evento LOGIN_OK ou LOGIN_FAIL

---

18. Ownership / IDOR Prevention

Descrição

Garante que um usuário jamais possa acessar ou modificar recursos de outro. Todos os endpoints retornam 404 (não 403) para recursos de outro usuário, evitando enumeração.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada

OWN-001	GET /transactions retorna apenas do userId	userA autenticado	Nenhum tx de userB
OWN-002	DELETE tx de outro usuário	tokenA, idTxB	404 (não 403)
OWN-003	PATCH tx de outro usuário	tokenA, idTxB	404
OWN-004	PATCH account de outro usuário	tokenA, idAccB	404
OWN-005	DELETE account de outro usuário	tokenA, idAccB	404
OWN-006	GET goal de outro usuário	tokenA, idGoalB	404
OWN-007	GET statements de CC de outro usuário	tokenA, idCCB	404
OWN-008	Usuário acessa os próprios recursos	tokenA, idTxA	200 ou 204

---

19. Faturamento & Planos

Descrição

Sistema de planos (FREE, PRO, ENTERPRISE) com feature gates. Usuários FREE têm limites de uso (transações, contas, etc.). Integração com Stripe para pagamentos.

Casos de Teste

ID	Cenário	Entrada	Saída Esperada
BIL-001	Usuário FREE tem plano correto	POST /register	currentPlan=FREE
BIL-002	Feature gate bloqueia FREE	tentar recurso PRO em conta FREE	403 com mensagem de upgrade
BIL-003	Limite de transações respeitado	FREE tenta criar além do limite	403
BIL-004	Checkout Stripe	POST /billing/checkout	retorna sessionUrl do Stripe
BIL-005	Webhook ativa PRO	simular webhook checkout.session.completed	currentPlan=PRO
BIL-006	GET /billing/status retorna plano atual	autenticado	currentPlan + limites

---

20. Matriz de Cobertura

Domínio	Unitário	Integração	Manual	Total Casos
Auth & Sessões	—	25	3	28

Transações	8 (recurring)	26	5	39
Contas	—	6	2	8
Categorias	—	7	1	8
Orçamentos	—	8	1	9
Cartões de Crédito	—	7	3	10
Metas Financeiras	9 (pure helpers)	3	2	14
Micro Metas	—	4	1	5
Passivos	—	5	1	6
Investimentos	—	5	2	7
Dashboard	—	4	2	6
Forecast	8 (daysInMonth + scenarios)	2	3	13
Regras de Categorização	—	5	1	6
Segurança de Dados	12 (encryption)	4	2	18
Ownership / IDOR	—	8	1	9
Faturamento	—	6	3	9
TOTAL	**37**	**125**	**33**	**195**

---

21. Critérios de Aprovação V0

Critério	Meta	Verificação
Testes unitários passando	100% (0 falhas)	`npm run test:unit`
Testes de integração passando	100% (0 falhas)	`npm run test:integration`
Cobertura de statements (serviços críticos)	≥ 80%	`npm run test:coverage`
Nenhum IDOR detectado	0 falhas nos 8 testes OWN-*	`npm run test:integration`
Auth flows completos	0 falhas nos 25 testes AUTH-*	`npm run test:integration`
Build TypeScript sem erros	0 erros	`npm run build`
Nenhum segredo em logs	0 ocorrências de password/token em logs	Revisão manual de logs
Todos os endpoints autenticados protegidos	Retornam 401 sem Bearer	Revisão de rotas

Execução do plano completo

```
# 1. Instalar dependências
cd backend && npm install

# 2. Preparar banco de testes
createdb financial_control_test
DATABASE_URL_TEST="postgresql://..." npx prisma migrate deploy

# 3. Configurar env de testes
echo
'DATABASE_URL_TEST="postgresql://fintrack:fintrack123@localhost:5432/financial_cont
> .env.test

# 4. Rodar todos os testes
npm run test:all

# 5. Gerar relatório de cobertura
npm run test:coverage

# 6. Build de produção
npm run build
```

Aprovado quando

```
✅ npm run test:unit      → X testes passaram, 0 falharam
✅ npm run test:integration → X testes passaram, 0 falharam
✅ npm run build          → Compiled successfully
✅ Coverage: Services     → Statements: ≥ 80%
```